

Flexible Load Balancing in Microservices

A Spring Boot & Docker Based Implementation
Group 12 — Microservices Architecture Course Project

Abstract

This paper presents the design and implementation of a Flexible Load Balancing system built using Spring Boot microservices architecture. The system demonstrates how incoming client requests can be distributed across multiple service instances using a Gateway service and Netflix Eureka for service discovery. The project implements three core microservices: a Eureka Server for service registry, a Product Service deployed in three instances, and a Gateway Service that performs load balancing. All services are containerized using Docker and orchestrated via Docker Compose. The results demonstrate successful round-robin and random load distribution across service instances, improving system availability and scalability.

1. Introduction

In modern software systems, especially those built on microservices architecture, handling high volumes of concurrent requests efficiently is a critical challenge. Load balancing is a technique used to distribute network or application traffic across multiple servers or service instances to ensure no single server bears too much demand. This improves responsiveness and increases the availability of applications.

This project aims to demonstrate a flexible load balancing implementation using the Spring Cloud ecosystem. By leveraging Netflix Eureka for service discovery and Spring Cloud Gateway for routing and load balancing, the system can dynamically distribute requests among available service instances without hardcoding their addresses.

2. Background

Microservices architecture structures an application as a collection of small, independently deployable services. Each service runs its own process and communicates through lightweight mechanisms. This architecture enables continuous delivery and deployment of large, complex applications.

Load Balancing Algorithms

Algorithm	Description	Use Case
Round Robin	Requests distributed equally in order	Equal capacity servers
Random	Random instance selection per request	Stateless services
Least Connection	Routes to instance with fewest connections	Varying request lengths

Weighted	Distributes based on server capacity weight	Heterogeneous servers
----------	---	-----------------------

Spring Cloud LoadBalancer is a client-side load balancer that integrates with Spring Cloud Gateway and Eureka. It supports both RoundRobinLoadBalancer and RandomLoadBalancer out of the box, and can be extended with custom strategies.

3. System Architecture

The system consists of three microservices working together:

Service	Port	Role
Eureka Server	8761	Service registry and discovery server
Product Service (x3)	8081, 8082, 8083	Business logic microservice with 3 instances
Gateway Service	8080	API gateway with load balancing capability

The request flow is as follows: A client sends an HTTP request to the Gateway Service on port 8080. The Gateway consults Eureka to obtain a list of available Product Service instances. Using the configured load balancing strategy (Random or Round Robin), it selects one instance and forwards the request. The selected Product Service processes the request and returns a response that includes its instance ID and port number, demonstrating which instance handled the request.

4. Development Details

4.1 Technology Stack

Technology	Version	Purpose
Java	21 (LTS)	Programming language
Spring Boot	3.2.0	Microservice framework
Spring Cloud	2023.0.0	Cloud-native features
Netflix Eureka	2.0.1	Service discovery
Spring Cloud Gateway	4.1.0	API gateway & routing
Spring Cloud LoadBalancer	4.1.0	Client-side load balancing
Docker	29.4.0	Containerization
Docker Compose	v2	Multi-container orchestration
Maven	3.9.14	Build tool

4.2 Eureka Server Implementation

The Eureka Server acts as the service registry. All microservices register themselves with Eureka on startup and send periodic heartbeats to maintain their registration. The server is configured with register-with-eureka: false and fetch-registry: false since it does not need to register with itself.

4.3 Product Service Implementation

The Product Service is a RESTful microservice that exposes two endpoints: /products for retrieving a product list, and /products/health for health checking. Each instance is configured with a unique INSTANCE_ID and SERVER_PORT environment variable injected via Docker Compose. The response includes the instance ID and port, making it easy to verify which instance handled each request.

4.4 Gateway Service Implementation

The Gateway Service uses Spring Cloud Gateway to route incoming requests. Routes are configured to forward /api/products/** to the product-service using the lb:// prefix, which enables client-side load balancing through Spring Cloud LoadBalancer. The StripPrefix filter removes the /api prefix before forwarding to the downstream service.

4.5 Docker Configuration

Each service has its own Dockerfile based on eclipse-temurin:21-jdk-alpine for a minimal image size. Docker Compose orchestrates all five containers (1 Eureka, 3 Product, 1 Gateway) within a shared bridge network called loadbalancing-network. The restart: on-failure policy ensures services automatically retry if they cannot connect to Eureka during startup.

5. Results & Conclusion

The implemented system successfully demonstrates flexible load balancing across three Product Service instances. When sending repeated GET requests to http://localhost:8080/api/products, the response alternates between product-instance-1, product-instance-2, and product-instance-3, confirming that the load balancer distributes traffic across all registered instances.

The Eureka Dashboard at http://localhost:8761 provides real-time visibility into registered services and their health status. The system demonstrates key microservices concepts including service discovery, dynamic routing, containerization, and horizontal scaling — all essential components of modern cloud-native applications.

References

- Spring Cloud Documentation — <https://spring.io/projects/spring-cloud>
- Netflix Eureka GitHub — <https://github.com/Netflix/eureka>
- Spring Cloud Gateway Reference — <https://docs.spring.io/spring-cloud-gateway>
- Docker Documentation — <https://docs.docker.com>
- Richardson, C. (2018). Microservices Patterns. Manning Publications.